

```
2590860 Finished chapter 1.  
2dece36 Started my novel.
```

Later you decide you want to start over again from the exciting plot twist at hash `8aa5222`. This means you should undo the changes from the last three commits: `de24642`, `2be4163`, and `97c655e`. Run `git revert -n HEAD~3..HEAD` to undo these changes, and then run `git add .` and `git commit -m "<commit message>"` to commit this content, just as you would with any other change:

```
C:\Users\Al\novel>git revert -n HEAD~3..HEAD  
  
C:\Users\Al\novel>git add .  
  
C:\Users\Al\novel>git commit -m "Starting over from the plot twist."  
[master faec20e] Starting over from the plot twist.  
1 file changed, 34 deletions(-)  
  
C:\Users\Al\novel>git log --oneline  
faec20e (HEAD -> master) Starting over from the plot twist.  
de24642 Changed the setting to outer space.  
2be4163 Added a whacky sidekick.  
97c655e Renamed the detective to 'Snuggles'.  
8aa5222 Added an exciting plot twist.  
2590860 Finished chapter 1.  
2dece36 Started my novel.
```

Git repos generally only add information, so undoing these commits still leaves them in the commit history. If you ever want to undo this “undo,” you can roll it back using `git revert` again.

Rolling Back to a Specific Commit for a Single File

Because commits capture the state of the entire repo instead of individual files, you’ll need a different command if you want to roll back changes for a single file. For example, let’s say I had a Git repo for a small software project. I’ve created an *eggs.py* file and added functions `spam()` and `bacon()`, and then renamed `bacon()` to `cheese()`. The log for this repo would look something like this:

```
C:\Users\Al\myproject>git log --oneline  
895d220 (HEAD -> master) Adding email support to cheese().
```